claude-windows / 902e27a6-e026-4b55-8c26-f2c47329cc44

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\902e27a6-e026-4b55-8c26-f2c47329cc44\subagents\workflows\wf_c287c071-8e9\agent-ab1e72897df2aae47.jsonl`
- SHA-256: `761242b66044c7719d88eeaa8679934c68c35c9a578f3bc1626ebbf228124997`
- Source modified: `2026-07-07T14:50:27+00:00`
- Imported at: `2026-07-08T16:00:02+00:00`
- project: `wf_c287c071-8e9`
- session_id: `902e27a6-e026-4b55-8c26-f2c47329cc44`

Transcript

1. user (2026-07-07T14:45:11.303Z)

```
You are reviewing an uncommitted change in a git worktree at:
  C:/Users/avidu/Projects/khelsutra-guru/bhi-wt-worker-reval
Branch: fix/worker-skip-revalidation-on-dispatch (based on origin/master).

THE BUG BEING FIXED: On the cloud-serving path, the control-plane (backend/orchestration.py)
runs
validators and ONLY dispatches to the Cloud Run worker on PASS. The worker
(backend/worker/__main__.py
cmd_infer) then REDUNDANTLY re-ran validation with its OWN config (different
min_blur_threshold),
rejecting a clip the control-plane already passed. Live failure 2026-07-07: passed at
min_blur_threshold=8.0 on the control-plane, worker rejected at its default 20.0 ? GPU dispatch
succeeded but the CUJ failed with worker rc=2.

THE FIX (skip-on-dispatch): a new ComputeJobSpec.skip_validation bool (default False) ?
backend/compute/cloud_run.py emits a "--skip-validation" worker CLI flag when set ?
backend/worker/__main__.py cmd_infer honors it (skips validator_registry.run_all_with_context,
sets val_results=[], val_context={}) ? backend/orchestration.py::_maybe_dispatch_remote sets
skip_validation=True on every dispatch. The direct "worker infer" CLI (no control-plane) still
validates by default. New tests in tests/test_worker.py and tests/test_api_cloud_dispatch.py.

Inspect the ACTUAL code ? do NOT trust this summary. Run:  cd
"C:/Users/avidu/Projects/khelsutra-guru/bhi-wt-worker-reval" && git --no-pager diff
and read the surrounding functions (not just the diff hunks). The full test suite already passes
(1779 passed), ruff + mypy are clean, coverage 85.42%. So do NOT report style nits or things the
gate already covers. Hunt for REAL correctness/rollout/consumer defects the tests would miss.
Only report a finding you can tie to a concrete failure scenario in THIS repo's code.

LENS: BACKWARD COMPATIBILITY & ROLLOUT SKEW. The "--skip-validation" flag is a NEW worker CLI
arg.
- If the deployed control-plane starts sending --skip-validation to a worker running an OLDER
image
  that doesn't define that argparse arg, what happens? (argparse behavior on unknown args.) Find
how
  the control-plane and worker images relate ? read deploy/cloudrun/*, gen_service_yaml.py, and
any
  docs on "ONE image drives both". Is there a rollout ordering that breaks (control-plane
updated
  before worker, or vice versa)? Compare to how #509 handled the same coupling for --decode-
backend /
  --set forwarding.
- Does adding skip_validation to the frozen ComputeJobSpec dataclass break anything (hashing,
existing
```

```
  positional constructions, pickling)?
Report concrete rollout/compat failure modes and whether a code guard (e.g. tolerating unknown
args)
or just a deploy-together release note is the right mitigation.
```

2. assistant (2026-07-07T14:45:15.498Z)

```
I'll start by examining the actual change and surrounding code.
```

3. user (2026-07-07T14:45:18.287Z)

```
diff --git a/backend/compute/base.py b/backend/compute/base.py
index 279606e..31af3fe 100644
--- a/backend/compute/base.py
+++ b/backend/compute/base.py
@@ -28,6 +28,13 @@ class ComputeJobSpec:
     segmenter: Optional[str] = None
     # Raw worker ``--set k=v`` overrides. A tuple (not list/dict) so the spec stays
frozen/hashable.
     config_overrides: tuple[str, ...] = ()
+    # Skip the worker's redundant re-validation. The DISPATCHER (the cloud-serving control-
plane) is
+    # the authoritative validation gate: it runs the SAME validators with ITS resolved config
and only
+    # dispatches on PASS, so a worker re-run is (a) redundant, (b) a second full-video decode,
and
+    # (c) outright WRONG when the worker's config resolves different thresholds ? it can REJECT
a clip
+    # the gate already passed. Default False keeps the direct ``worker infer`` CLI (no control-
plane)
+    # validating exactly as before.
+    skip_validation: bool = False


 @dataclass(frozen=True)
diff --git a/backend/compute/cloud_run.py b/backend/compute/cloud_run.py
index 841785d..7247ace 100644
--- a/backend/compute/cloud_run.py
+++ b/backend/compute/cloud_run.py
@@ -196,6 +196,10 @@ class CloudRunCompute(ComputeBackend):
         ]
         if spec.segmenter:
             argv += ["--segmenter", spec.segmenter]
+        if spec.skip_validation:
+            # The control-plane already validated + only dispatches on PASS ? tell the worker
not to
+            # re-decode and re-validate (redundant, and its config may resolve different
thresholds).
+            argv.append("--skip-validation")
         for kv in (*spec.config_overrides, *self._container_overrides):
             argv += ["--set", kv]

diff --git a/backend/orchestration.py b/backend/orchestration.py
index 5e9f510..79b4fed 100644
--- a/backend/orchestration.py
+++ b/backend/orchestration.py
@@ -568,11 +568,18 @@ def _maybe_dispatch_remote(
         prefetch = getattr(wasb_cfg, "prefetch_batches", None)
         if prefetch:
             overrides.append(f"indexing.wasb.prefetch_batches={prefetch}")
+    # Tell the worker to SKIP its redundant re-validation: THIS control-plane is the
authoritative
+    # gate ? it already ran the validators with its resolved config just above (only a PASS
reaches
+    # here; a failure returned "failed" before dispatch). A worker re-run would decode the
```

```
video a
+    # second time and, because its config isn't fully forwarded, can resolve DIFFERENT
thresholds and
+    # REJECT a clip this gate already passed (observed 2026-07-07: passed at
min_blur_threshold=8.0,
+    # then worker-rejected at its default 20.0 after a successful GPU dispatch).
     spec = ComputeJobSpec(
         video_uri=req.video_path,
         out_uri=out_root,
         segmenter=seg_name,
         config_overrides=tuple(overrides),
+        skip_validation=True,
     )
     try:
         # Dispatch the Cloud Run Job + bucket-poll to completion. on_wait is the live
diff --git a/backend/worker/__main__.py b/backend/worker/__main__.py
index 88c50f6..9b19215 100644
--- a/backend/worker/__main__.py
+++ b/backend/worker/__main__.py
@@ -186,6 +186,7 @@ def _dispatch_remote(compute: Any, args: argparse.Namespace) -> int:
         out_uri=args.out_uri,
         segmenter=args.segmenter,
         config_overrides=tuple(args.set or ()),
+        skip_validation=bool(getattr(args, "skip_validation", False)),
     )
     try:
         result = compute.run_infer(spec)
@@ -259,10 +260,25 @@ def cmd_infer(args: argparse.Namespace) -> int:
     # 2. IDENTITY ? whole-file MD5 (the labels?video join key; pulled bytes must be byte-
identical).
     video_id = compute_video_id(local_video)

-    # 3. VALIDATE (same registry as the main CLI).
-    val_results, val_context = validator_registry.run_all_with_context(
-        local_video, config
-    )
+    # 3. VALIDATE (same registry as the main CLI) ? UNLESS the dispatcher already did.
+    # On the cloud-serving path the CONTROL-PLANE is the authoritative validation gate: it runs
the
+    # SAME validators with ITS resolved config and only dispatches on PASS. Re-validating here
is
+    # redundant, decodes the whole video a SECOND time, and is outright WRONG when the worker's
config
+    # resolves different thresholds than the control-plane's ? observed 2026-07-07: a 5312x2988
clip
+    # (sharpness 11.9) PASSED the control-plane at validation.min_blur_threshold=8.0, then the
worker
+    # REJECTED it at its own default 20.0, so a SUCCESSFUL GPU dispatch failed after the fact.
+    # --skip-validation (set by the dispatcher in orchestration._maybe_dispatch_remote) skips
it; the
+    # direct ``worker infer`` CLI has no control-plane, so it still validates by default.
+    if getattr(args, "skip_validation", False):
+        print(
+            "[worker] validation SKIPPED (--skip-validation): the dispatcher is the
authoritative "
+            "gate and already validated this input."
+        )
+        val_results, val_context = [], {}
+    else:
+        val_results, val_context = validator_registry.run_all_with_context(
+            local_video, config
+        )
     failures = [r for r in val_results if not r.passed]
     db = Database(os.path.join(scratch, config.output.db_name))
     db.add_video(
```

```
@@ -724,6 +740,14 @@ def build_parser() -> argparse.ArgumentParser:
         metavar="k=v",
         help="config override, e.g. --set indexing.skip_ai_handoff=true",
     )
+    pi.add_argument(
+        "--skip-validation",
+        action="store_true",
+        help="skip the worker's re-validation: the dispatcher (control-plane) already ran the "
+        "validators with its resolved config and only dispatches on PASS, so re-validating here
is "
+        "redundant, decodes the video again, and can contradict the gate. Default-OFF: the
direct "
+        "`worker infer` CLI still validates.",
+    )
     pi.set_defaults(func=cmd_infer)

     pt = sub.add_parser(
diff --git a/tests/test_api_cloud_dispatch.py b/tests/test_api_cloud_dispatch.py
index 8382234..19c37d9 100644
--- a/tests/test_api_cloud_dispatch.py
+++ b/tests/test_api_cloud_dispatch.py
@@ -236,6 +236,22 @@ def test_cloud_dispatch_omits_unset_wasb_tuning(cloud_env):
     assert "indexing.wasb.prefetch_batches" not in joined


+def test_cloud_dispatch_skips_worker_revalidation(cloud_env):
+    """The control-plane is the AUTHORITATIVE validation gate: it validated + only dispatches
on PASS
+    (registry.run_all_with_context is stubbed to pass in cloud_env). Re-validating on the
worker is
+    redundant, decodes the video a second time, and ? because the control-plane's validation
config
+    isn't forwarded ? can REJECT a clip the gate already passed (observed 2026-07-07: passed at
+    validation.min_blur_threshold=8.0, then worker-rejected at its default 20.0). So the
dispatch MUST
+    carry --skip-validation, telling the worker to trust the gate. Regression guard for that
wiring."""
+    _db, _bucket, monkeypatch = cloud_env
+    fake = _FakeCloudClient()
+    _inject_fake(monkeypatch, fake)
+    m._execute_processing(
+        m.ProcessRequest(video_path="gs://b/clip.mp4", segmenter="wasb_hybrid")
+    )
+    assert "--skip-validation" in fake.calls[0]
+
+
 def test_cloud_failure_marks_failed_no_rows(cloud_env):
     db, _bucket, monkeypatch = cloud_env
     # pre-seed a prior good row ? must be preserved on a cloud failure (destroy-AFTER-confirm).
diff --git a/tests/test_worker.py b/tests/test_worker.py
index 8fb5e16..72b2514 100644
--- a/tests/test_worker.py
+++ b/tests/test_worker.py
@@ -97,6 +97,14 @@ def test_parser_infer_accepts_set_and_uris():
     assert args.set == ["indexing.skip_ai_handoff=true", "sport=tennis"]


+def test_parser_infer_skip_validation_flag():
+    base = ["infer", "--video-uri", "s3://b/v.mp4", "--out-uri", "s3://b"]
+    # Default-OFF: the direct `worker infer` CLI still validates.
+    assert wmain.build_parser().parse_args(base).skip_validation is False
+    # The dispatcher passes it to disable the redundant worker re-validation.
+    assert wmain.build_parser().parse_args(base + ["--skip-validation"]).skip_validation is
True
+
```

```
+
 # ------------------------------------------------------------ cmd_infer (local, mocked)


@@ -407,6 +415,43 @@ def test_cmd_infer_validation_failure_returns_2(tmp_path, monkeypatch):
     assert marker["returncode"] == 2 and marker["status"] == "failed"


+def test_cmd_infer_skip_validation_bypasses_validators(tmp_path, monkeypatch):
+    """--skip-validation (set by the dispatcher, the authoritative gate) must NOT run the
worker's
+    validators ? no second decode, and no chance of a threshold mismatch REJECTING a clip the
+    control-plane already passed (the live 2026-07-07 CUJ failure). Prove it by making
+    run_all_with_context RAISE: a skipped run never touches it, so the job completes rc 0 with
a
+    status="success" report. Contrast test_cmd_infer_validation_failure_returns_2 (no flag ? rc
2)."""
+    monkeypatch.setattr(wmain, "compute_video_id", lambda p: "vidSkip")
+
+    def _boom(*a, **k):  # the validators must never be reached on the skip path
+        raise AssertionError("run_all_with_context must not run under --skip-validation")
+
+    monkeypatch.setattr(wmain.validator_registry, "run_all_with_context", _boom)
+    monkeypatch.setattr(wmain.segmenter_registry, "get", lambda name: _FakeSeg)
+    monkeypatch.setattr(wmain, "emit_indexer_report", lambda **k: None)
+    video = tmp_path / "in.mp4"
+    video.write_bytes(b"FAKE")
+    out = tmp_path / "bucket"
+
+    rc = wmain.main(
+        [
+            "infer",
+            "--video-uri",
+            str(video),
+            "--out-uri",
+            str(out),
+            "--scratch",
+            str(tmp_path / "s"),
+            "--segmenter",
+            "wasb_hybrid",
+            "--skip-validation",
+        ]
+    )
+    assert rc == 0  # a clip that WOULD fail re-validation still succeeds ? the gate is trusted
+    report = json.loads((out / "outputs" / "vidSkip" / "report.json").read_text())
+    assert report["status"] == "success" and report["segment_count"] == 2
+
+
 # --- cmd_train: pull labels -> synth trajectories -> manifest -> harness -> push bundle ---
```

4. user (2026-07-07T14:45:19.240Z)

```
3be0935 docs(runlog): DEPLOY_REPORT addendum ? control-plane redeployed to the forwarding image
(#510)
66756ad chore(serving): forward WASB decode/tuning knobs to worker + release hardening (#509)
143dbb9 feat(serving): default cloud-serving to nvdec_resident decode (#508)
44a2f47 feat(detectors): GPU-resident WASB NVDEC decode behind default-OFF decode_backend flag
(#507)
17d844b docs(runlog): alpha-tester "All 1 v 1" cloud-serving run ? timeout diagnosed +
downsample fix proven (#504)
bd5647b fix(worker): surface detector failure/timeout as failed, not silent 0-rally success
(#503)
07e13f7 docs: Alpha Tester Onboarding FAQ ? send-ready copy for the first invite wave (#499)
2a8a931 docs: record the owner's §1.1 browser CUJ PASS ? DoD #1 satisfied (rev 00015-kp9) (#498)
---STATUS---
```

```
On branch fix/worker-skip-revalidation-on-dispatch
Your branch is up to date with 'origin/master'.

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   backend/compute/base.py
        modified:   backend/compute/cloud_run.py
        modified:   backend/orchestration.py
        modified:   backend/worker/__main__.py
        modified:   tests/test_api_cloud_dispatch.py
        modified:   tests/test_worker.py

no changes added to commit (use "git add" and/or "git commit -a")
```

5. assistant (2026-07-07T14:45:28.254Z)

Let me examine the deploy setup, how #509 handled coupling, and the worker's argparse behavior.

6. user (2026-07-07T14:45:32.135Z)

```
cloudbuild.yaml
Dockerfile
gen_service_yaml.py
post_deploy_smoke.ps1
README.md
---
cloudrun
digitalocean
gcp
---gen_service---
./deploy/cloudrun/gen_service_yaml.py
```

7. user (2026-07-07T14:45:33.299Z)

```
commit 66756adfc6573d85ef884787d6653332db31201a
Author: Avi <avi.dullu@gmail.com>
Date:   Tue Jul 7 18:29:44 2026 +0530

    chore(serving): forward WASB decode/tuning knobs to worker + release hardening (#509)

    Makes the cloud-serving preset the source of truth on the dispatch path (control plane now
forwards indexing.wasb.{decode_backend,batch_size,prefetch_batches} to the worker Job, which has
no DEPLOYMENT_PROFILE), + .gcloudignore, a reusable post_deploy_smoke.ps1, and deploy-doc
updates. Activating needs a control-plane redeploy; WASB_DECODE_BACKEND keeps nvdec live until
then.

 .gcloudignore                                      | 20 +++++++
 backend/orchestration.py                           | 20 ++++++-
 deploy/cloudrun/README.md                          | 44 ++++++++++++-
 deploy/cloudrun/post_deploy_smoke.ps1              | 67 ++++++++++++++++++++++
 ..._cloud-serving_gpu-resident-nvdec_2026-07-07.md |  2 +-
 tests/test_api_cloud_dispatch.py                   | 37 ++++++++++++
 6 files changed, 185 insertions(+), 5 deletions(-)
```

8. user (2026-07-07T14:45:43.853Z)

```
1       # Cloud Run GPU serving ? build / deploy / run (COMPUTE_DECOUPLED_SERVING M6/M7)
2
3       The cloud-serving compute path: a control plane (`CloudRunCompute`,
`backend/compute/cloud_run.py`)
4       dispatches a Cloud Run **Job** execution running this image's worker on an **L4**, then
**bucket-polls**
5       a done-marker for completion (D16). Scale-to-zero (D7) ? no warm pool.
```

```
6
7       > **Status:** the dispatch shim + image are **landed + mock-tested** (M6). The **real
build + push +
8       > run on an L4** is the **M7** owner step, within the **$100/?10k** budget (D17) ?
confirm cmd + hourly
9       > cost first, **delete every resource after** ([[gcp-vm-always-delete]]). CI does
**not** build this image.
10
11      ## 0. Prerequisites (owner)
12      - A GCP project with billing on + the Cloud Run, Artifact Registry, and Cloud Build APIs
enabled.
13      - GPU quota for L4 (`GPUS_ALL_REGIONS` ? 1) in the chosen region (asia-south1, else
Singapore ? see
14        `deploy/gcp/README.md`; L4 is often stocked-out, fall back per that runbook).
15      - The WASB weights staged in the bucket (WASB-C3 unresolved ? owner-gated):
`gs://<bucket>/models/wasb_badminton_best.pth.tar`.
16
17      ## 1. Build + push the image (Artifact Registry)
18      ```bash
19      PROJECT=<gcp-project>; REPO=rally
20      # L4 GPU is region-limited ? use asia-southeast1 (Singapore), NOT asia-south1 (Mumbai).
21      REGION=asia-southeast1
22      gcloud artifacts repositories create $REPO --repository-format=docker --location=$REGION
2>/dev/null || true
23      # `gcloud builds submit` has NO -f flag, so a non-root Dockerfile needs a build config:
24      gcloud builds submit --config=deploy/cloudrun/cloudbuild.yaml --project $PROJECT .
25      ```
26      *(A CUDA + micromamba image is large; the first build is ~8?10 min. Budget-watch per
D17.)*
27
28      ## 2. Create the Cloud Run **Job** (GPU, scale-to-zero)
29      The proven shape (first real M7 deploy, 2026-07-03): weights staged in a **same-region
bucket**
30      mounted read-only via GCS FUSE (no weights in the image ? WASB-C3; no init-fetch step
needed).
31      ```bash
32      gcloud run jobs create rally-worker \
33        --image $REGION-docker.pkg.dev/$PROJECT/$REPO/$IMG:latest \
34        --region $REGION --gpu 1 --gpu-type nvidia-l4 --no-gpu-zonal-redundancy \
35        --cpu 8 --memory 32Gi \
36        --max-retries 0 --task-timeout 3600 \
37        --set-env-vars WASB_WEIGHTS=/gcs/models/wasb_badminton_best.pth.tar \
38        --add-volume name=serving,type=cloud-storage,bucket=$SERVING_BUCKET \
39        --add-volume-mount volume=serving,mount-path=/gcs
40      ```
41      - **`--no-gpu-zonal-redundancy` is REQUIRED on the CLI**, not just a cost choice (it
also halves
42        the GPU rate ? right for batch jobs): without the flag `gcloud` asks an interactive
Y/n which,
43        in a non-interactive shell, **hangs forever with zero output and nothing created**. An
empty
44        `gcloud` that "did nothing" ? suspect a hidden prompt.
45      - 8 vCPU / 32 GiB: x264 encode is CPU-side, so CPU headroom keeps the paid GPU busy.
`/tmp` is
46        **RAM-backed** ? the pulled input video lives in memory, so size `--memory` for the
largest
47        expected upload (a 5.8 GB GoPro file ? 6 GiB of tmpfs).
48
49      ## 2a. GPU-resident WASB decode (#506/#507) ? the current serving default
50      The image ships the **py3.10 + torch 2.6.0+cu124 + PyNvVideoCodec==2.1.0** WASB env with
51      `libnvcuvid`/`libnvidia-encode` **baked in** (multi-stage layer from the matching NVIDIA
driver ?
52      Cloud Run injects the host driver, not these userspace codec libs).
`indexing.wasb.decode_backend`
53      selects the decode path:
```

```
- `cpu` (model default) ? the historical cv2 decode; **byte-unchanged** by this work.
- `nvdec_resident` ? NVDEC decode + cv2-faithful affine `grid_sample` resize + GPU
normalize, all
  on-GPU. **The cloud-serving preset default** (validated: offline golden gate @ SERVED
F1 0.574 ?
  baseline 0.582, AND on the live L4 the served GX010128 trajectory reproduced the VM
run ?
  26 479 ? 26 481 visible). CUDA-only (config-validator-enforced).

**How the knob reaches the worker.** The worker Job runs with NO `DEPLOYMENT_PROFILE`,
so it does
**not** apply the cloud-serving preset itself. The control plane (which *does* have the
profile)
resolves the config and **forwards**
`indexing.wasb.{decode_backend,batch_size,prefetch_batches}`
to the worker as `--set` overrides (`orchestration.py`, alongside `skip_ai_handoff`). So
the
**preset is the single source of truth** ? changing `presets.py` + redeploying the
control plane
changes serving; no per-Job knob needed.

**Override / rollback lever (no redeploy).** The native runner reads
`WASB_DECODE_BACKEND` FIRST,
so a Job env var wins over the forwarded `--set`:
```bash
# force nvdec (or pin cpu) on the Job regardless of the forwarded config; instant
rollback = remove it
gcloud run jobs update rally-worker --region $REGION --update-env-vars
WASB_DECODE_BACKEND=nvdec_resident
gcloud run jobs update rally-worker --region $REGION --remove-env-vars
WASB_DECODE_BACKEND   # ? forwarded/preset default
# full revert: pin the prior image by digest
gcloud run jobs update rally-worker --region $REGION --image $REGION-
docker.pkg.dev/$PROJECT/$REPO/rally-worker@sha256:<prev>
```
? Forwarding `decode_backend=nvdec_resident` requires a worker image that understands
`--decode-backend` (? #507). Roll the **worker image and the control plane together** ?
a new
control plane forwarding nvdec to an old worker image argparse-fails (rc 2).

Post-deploy smoke. After a cutover, confirm the real L4 path with
`deploy/cloudrun/post_deploy_smoke.ps1` (run from PowerShell ? see §Gotchas on MSYS arg-
mangling):
```powershell
./deploy/cloudrun/post_deploy_smoke.ps1 -Video gs://khelsutra-rally-
corpus/videos/mahadevpura_smoke_180s.mp4 -Decode cpu    # image sanity (fast)
./deploy/cloudrun/post_deploy_smoke.ps1 -Video gs://khelsutra-rally-
corpus/videos/GX010128.MP4       -Decode env    # nvdec via preset/env (hard clip)
```

3. Wire the dispatcher (control plane)
The control plane runs with the **cloud-serving** profile (`DEPLOYMENT_PROFILE=cloud-
serving` ?
`compute_target=cloud_run`). `get_compute_backend(config)` then builds `CloudRunCompute`
from env:
```bash
export DEPLOYMENT_PROFILE=cloud-serving
export CLOUD_RUN_JOB=rally-worker  CLOUD_RUN_REGION=$REGION
GOOGLE_CLOUD_PROJECT=$PROJECT
# A `worker infer` (or the API job path) with a gs:// video now DISPATCHES to the Job
instead of
# running in-process; the container runs the SAME worker INLINE (compute_target forced
local_gpu).
python -m backend.worker infer --video-uri gs://<bucket>/videos/clip.mp4 --out-uri
gs://<bucket> --segmenter wasb_hybrid
```
```

```
**The AI handoff needs its key IN THE JOB env** (`GEMINI_API_KEY` ? mount a Secret Manager secret
on the job), or run fully-local with `--set indexing.skip_ai_handoff=true` (candidate windows
emitted as rallies ? the eval-parity mode). With neither, the segmenter used to bail to a silent
0-rally "success" **after** the paid GPU pass; the cloud-serving startup checks now fail this at
boot (`AI_HANDOFF_KEY_MISSING`, rc 2) before the video is even pulled.

### 3a. Control-plane SERVICE deploy ? required flags for reliable async jobs

The API server (`/api/process`, `/api/compile` ? #329) runs its **job worker IN-PROCESS**: the
request returns `202` and a background thread drains the job (GPU detection dispatches to the Job;
the reel stitch runs on the control plane itself). On Cloud Run that background thread only makes
progress while the instance has CPU, so a scale-to-zero **service** deploy MUST set:

**The canonical deploy path is the committed manifest generator** (#473) ? it bakes the
cloud-serving config.json into a startup wrapper, sets every flag below, pins `maxScale=1`
(per-instance SQLite state, issue #471), and deliberately does NOT set `CLOUD_RUN_JOB` (a
reserved runtime env name that gets a manifest rejected; the code default is already right):
```bash
python deploy/cloudrun/gen_service_yaml.py --edge-auth on -o service.yaml
gcloud run services replace service.yaml --region $REGION   # from PowerShell on Windows
```
`--edge-auth off` is only for the pre-CF identity-token smoke/e2e window; redeploy with `on`
before testers touch the service again. The flag-based equivalent (for reference ? a manifest
round-trips the bash wrapper's metacharacters, flags don't on Windows):
```bash
gcloud run deploy rally-control-plane --image <image> --region $REGION \
  --no-cpu-throttling \        # CPU stays allocated between requests ? the drain thread finishes
  --min-instances 1 \          # one always-warm instance drains reliably (cheap CPU; NOT the D7 GPU pool)
  --cpu 2 --memory 4Gi --port 8080 --allow-unauthenticated \
  --set-env-vars DEPLOYMENT_PROFILE=cloud-serving,CLOUD_RUN_REGION=$REGION,GOOGLE_CLOUD_PROJECT=$PROJECT
```
Without `--no-cpu-throttling`, a `/api/compile` returns 202 and then **stalls** ? the reel never
finishes and startup crash-recovery would mark it terminal. `min-instances 1` does NOT contradict
D7 (that was the ~$500/mo warm **GPU** pool); this is a ~1-vCPU CPU instance. Defense-in-depth is in
the code: `Database.recover_stale_jobs()` **re-queues** a compile interrupted by a restart (idempotent
stitch) rather than failing it, so even an unlucky scale-down mid-stitch self-heals on the next drain.
Reserve `--allow-unauthenticated` for the pre-CF-auth smoke test only; lock ingress to the CF proxy
(M9) before inviting testers. The image ships the `auth` extra (PyJWT[crypto]), so flipping
`security.edge_auth_enabled` on needs no derived image ? the M9 Cf-Access JWT verify works out of
the box.
```

```
138     ## 4. M7 acceptance (the owner runlog)
139     Capture a `DEPLOY_REPORT_cloud-serving_<date>.md` under
140     `docs/COMPUTE_DECOUPLED_SERVING/runlogs/` (mirrors the truly-local template): the exact
commands,
141     rally counts, `gpu_active_seconds`/`wall_seconds`/`bytes_out`, a reel smoke-test
(`ffmpeg -f null -`
142     exit 0 + a 5-frame contact sheet), gotchas, and the **$0 teardown audit**.
143
144     ## Gotchas
145     - The dispatched container **must run inline** ? `CloudRunCompute` forces
`compute_target=local_gpu`
146       + `indexing.detector_impl=native` in the per-execution args so the Job never re-
dispatches itself.
147     - Completion is a **bucket marker** (`<out_uri>/_dispatch/<token>.done`), not the Cloud
Run API status
148       ? so it reuses the storage layer + `execution_policy.poll_until` and survives a
control-plane restart.
149     - A **failed** Job (validation rc 2 / segmenter rc 3) now writes a **failure marker**
carrying the
150       real non-zero rc, so the dispatcher's poll terminates **FAST + accurately** (no 30-min
hang on a bad
151       video). Only a **hung/crashed** container (no marker at all) makes the bounded poll
time out
152       (`ExecutionPolicy.max_wait_sec`); the worker CLI maps that to a clean **rc 4**, and
the API path
153       surfaces it as a `cloud dispatch error` job failure (not a traceback).
154     - libx264 encode (parity, D16) + NVDEC decode-only; the image keeps ffmpeg's default
libx264.
155     - **`GOOGLE_CLOUD_PROJECT` is required** for the real client ?
`GoogleCloudRunJobClient.run_job`
156       fails fast with a clear `RuntimeError` if it is unset/empty (it must resolve a real
`job_path`;
157       an unset project would otherwise build a bogus `projects/None/...` name). Export it
(step 3) before
158       dispatching. The guard runs before the `google-cloud-run` import, so it is exercised
in CI even
159       though the SDK isn't a CI/test dependency.
160     - **Hand-rolled `jobs execute --args` with `gs://` URIs:** gcloud's default comma-split
plus its
161       alternate-delimiter syntax both bite ? repeated `--set` tokens trip the comma parser,
and a
162       `^:^` delimiter splits the `:` inside `gs://`. Use a delimiter that appears nowhere in
the args,
163       e.g. `--args='^@@^-v@@infer@@--video-uri=gs://?'`. (The dispatcher path doesn't have
this
164       problem ? prefer it; this is for one-off diagnostics.) Container-arg overrides must
include the
165       dispatcher's two forced `--set`s: `deployment.compute_target=local_gpu`,
166       `indexing.detector_impl=native`.
167     - **Git Bash (MSYS) on Windows mangles Unix-looking values** before gcloud sees them:
168       `--add-volume-mount ?mount-path=/gcs` became `C:/Program Files/Git/gcs`, and
169       `--command=/bin/bash` became `C:/Program Files/Git/usr/bin/bash` (the container then
exec-fails).
170       Run gcloud commands with such flags from PowerShell (`MSYS_NO_PATHCONV=1` breaks
gcloud's own
171       wrapper). PowerShell has its own trap: quote any flag value containing commas.
172     - **First execution after an image push** spends minutes in "Container image import in
progress"
173       (visible in `executions describe status.conditions`, NOT in logs) ? that's progress,
not a hang;
174       later executions reuse the imported image.
175
```

9. user (2026-07-07T14:45:49.116Z)

```
"""Generate the control-plane Cloud Run *service manifest* (COMPUTE_DECOUPLED_SERVING
M8/M9, #473).

The A1 alpha deploy drove `rally-control-plane` from a hand-generated YAML that lived in
a
session scratchpad ? this commits the generator so the deploy is reproducible from a
clean
clone. It emits a Knative service manifest for::

    python deploy/cloudrun/gen_service_yaml.py --edge-auth off -o service.yaml
    gcloud run services replace service.yaml --region asia-southeast1

Why a MANIFEST (`services replace`) and not `gcloud run deploy` flags: the container
command is
a /bin/bash wrapper carrying shell metacharacters (pipes, redirects, &&) ? on Windows
the gcloud
cmd-shim mangles those through `--command/--args`, while a YAML round-trips them
verbatim. Run
gcloud from PowerShell, not git-bash (MSYS rewrites `/tmp`-style paths).

The app is configured through a config.json baked INTO the manifest: the wrapper
base64-decodes
it to `/tmp/apphome/config.json` and exports `RALLY_APP_HOME=/tmp/apphome` before
exec'ing the
server, so one image serves every config shape (config.json wins over the cloud-serving
preset ?
backend/config/loader.py). `--edge-auth` is REQUIRED and explicit: `off` is only for the
identity-token smoke/e2e window; redeploy with `on` (the Cf-Access JWT verify, M9)
before the
service faces testers again.

Deliberate manifest choices (learned on the A1 deploy):
  * `run.googleapis.com/cpu-throttling: "false"` + minScale 1 ? the in-process job
worker drains
    on a background thread; without always-allocated CPU a 202'd compile stalls (README
§3a).
  * maxScale DEFAULTS TO 1 ? the jobs/videos DB is per-instance ephemeral SQLite, so a
second
    instance splits the queue and `/api/jobs/{id}` polls (issue #471, option 1). Raise
it only
    once state is shared.
  * `CLOUD_RUN_JOB` is NOT set ? it's a reserved runtime env name and the manifest is
rejected
    with it; the code default ("rally-worker") is already correct.
  * No secrets here: bucket/project/aud/team-domain are the same non-secret identifiers
the
    committed runlogs carry; the Gemini key stays in Secret Manager (the alpha runs
    `skip_ai_handoff=true` and never needs it).

Baked config pins (CONTROL_PLANE_DURABLE_REBUILD P1/D5 ? the live rev-00011 posture,
learned on
the 2026-07-05 owner CUJ; before this, regenerating the manifest silently DROPPED both):
  * `indexing.default_segmenter=wasb_hybrid` ? the engine default is yolo_hybrid, so an
unpinned
    cloud job ran the wrong segmenter (#479).
  * `security.max_part_mb=24` ? upload parts must fit under the ~32 MiB HTTP/1 body cap
at the
    Google Front End (#480); the engine default (95) gets every part 413'd before
FastAPI runs.
    NOT fixed via `use-http2`: h2c requires the container itself to speak cleartext
HTTP/2,
    which uvicorn does not (spec D6) ? the annotation would break the service.
  * `--memory` defaults to 8Gi (spec D2) ? compile still pulls the gs:// source into the
    RAM-backed /tmp to stitch, so the instance must hold the largest allowed upload with
    headroom; >8Gi would force 4 vCPU on Cloud Run for no current need.
```

```
    """

    from __future__ import annotations

    import argparse
    import base64
    import json
    import sys

    DEFAULT_PROJECT = "gen-lang-client-0306026826"
    DEFAULT_REGION = "asia-southeast1"
    DEFAULT_SERVICE = "rally-control-plane"
    DEFAULT_SERVING_BUCKET = "gs://khelsutra-rally-serving"
    DEFAULT_CF_TEAM_DOMAIN = "https://khelsutra.cloudflareaccess.com"
    DEFAULT_CF_ACCESS_AUD =
"c980da67ed888fff935fcd913ae1eb4da63f9d65805d6e1c5ca76f92f517082b"


    def build_config(args: argparse.Namespace) -> dict:
        """The app config the wrapper materializes at RALLY_APP_HOME/config.json."""
        bucket = args.serving_bucket.rstrip("/")
        return {
            "deployment": {"profile": "cloud-serving", "compute_target": "cloud_run"},
            "indexing": {
                # WASB-only alpha: candidate windows ARE the rallies; no Gemini key in this
service.
                "skip_ai_handoff": True,
                # rev-00011 pin (#479): the engine default is yolo_hybrid ? pin the
validated one.
                "default_segmenter": args.default_segmenter,
            },
            "storage": {
                "backend": "gcs",
                "output_root": bucket,
                "input_backend": "gcs",
                "input_root": f"{bucket}/videos",
            },
            "security": {
                "edge_auth_enabled": args.edge_auth == "on",
                "cf_team_domain": args.cf_team_domain,
                "cf_access_aud": args.cf_access_aud,
                "allowed_video_root": "/tmp/apphome/output",
                # rev-00011 pin (#480): parts must clear the ~32 MiB HTTP/1 cap at the GFE.
                "max_part_mb": args.max_part_mb,
            },
        }


    def build_yaml(args: argparse.Namespace) -> str:
        """Render the Knative service manifest for `gcloud run services replace`."""
        cfg_b64 = base64.b64encode(
            json.dumps(build_config(args), separators=(",", ":")).encode("utf-8")
        ).decode("ascii")
        # mkdir BEFORE the redirect (the redirect can't create the directory), then exec so
the
        # server is PID 1 and Cloud Run's SIGTERM reaches it directly.
        wrapper = (
            "mkdir -p /tmp/apphome && "
            f"printf %s '{cfg_b64}' | base64 -d > /tmp/apphome/config.json && "
            "export RALLY_APP_HOME=/tmp/apphome && "
            "exec python3 -m backend.main --server --host 0.0.0.0 --port 8080"
        )
        return f"""\
    apiVersion: serving.knative.dev/v1
    kind: Service
```

```
    metadata:
      name: {args.service}
      labels:
        cloud.googleapis.com/location: {args.region}
    spec:
      template:
        metadata:
          annotations:
            autoscaling.knative.dev/minScale: "{args.min_instances}"
            autoscaling.knative.dev/maxScale: "{args.max_instances}"
            run.googleapis.com/cpu-throttling: "false"
        spec:
          timeoutSeconds: {args.timeout}
          containers:
            - image: {args.image}
              command: ["/bin/bash"]
              args: ["-c", "{wrapper}"]
              ports:
                - containerPort: 8080
              resources:
                limits:
                  cpu: "{args.cpu}"
                  memory: {args.memory}
              env:
                - name: DEPLOYMENT_PROFILE
                  value: cloud-serving
                - name: CLOUD_RUN_REGION
                  value: {args.region}
                - name: GOOGLE_CLOUD_PROJECT
                  value: {args.project}
                - name: RALLY_ALLOWED_HOSTS
                  value: "*"
    """


    def main(argv: "list[str] | None" = None) -> int:
        p = argparse.ArgumentParser(description=__doc__.splitlines()[0])
        p.add_argument(
            "--edge-auth",
            choices=("on", "off"),
            required=True,
            help="REQUIRED and explicit: 'off' only for the identity-token smoke/e2e window;
"
            "redeploy 'on' (M9 Cf-Access verify) before testers touch it again.",
        )
        p.add_argument("--project", default=DEFAULT_PROJECT)
        p.add_argument("--region", default=DEFAULT_REGION)
        p.add_argument("--service", default=DEFAULT_SERVICE)
        p.add_argument(
            "--image",
            default=None,
            help="Container image; default = the project's rally-worker:latest (post-#470 it
"
            "carries cloud-run + auth, so the worker image IS the control-plane image).",
        )
        p.add_argument("--serving-bucket", default=DEFAULT_SERVING_BUCKET)
        p.add_argument("--cf-team-domain", default=DEFAULT_CF_TEAM_DOMAIN)
        p.add_argument("--cf-access-aud", default=DEFAULT_CF_ACCESS_AUD)
        p.add_argument(
            "--default-segmenter",
            default="wasb_hybrid",
            help="rev-00011 pin (#479): the engine default is yolo_hybrid; the alpha serves
the "
            "validated wasb_hybrid.",
        )
```

```
168         p.add_argument(
169             "--max-part-mb",
170             type=int,
171             default=24,
172             help="rev-00011 pin (#480): upload parts must fit under Cloud Run's ~32 MiB
HTTP/1 "
173             "request-body cap (the engine default 95 gets 413'd at the edge).",
174         )
175         p.add_argument("--min-instances", type=int, default=1)
176         p.add_argument(
177             "--max-instances",
178             type=int,
179             default=1,
180             help="Default 1: per-instance SQLite state splits across instances (issue
#471).",
181         )
182         p.add_argument("--cpu", type=int, default=2)
183         p.add_argument(
184             "--memory",
185             default="8Gi",
186             help="Default 8Gi (CONTROL_PLANE_DURABLE_REBUILD D2): compile pulls the gs://
source "
187             "into RAM-backed /tmp, so size for the largest allowed upload plus headroom.",
188         )
189         p.add_argument("--timeout", type=int, default=300)
190         p.add_argument("-o", "--output", default="-", help="Output path, or '-' for
stdout.")
191         args = p.parse_args(argv)
192         if args.image is None:
193             args.image = f"{args.region}-docker.pkg.dev/{args.project}/rally/rally-
worker:latest"
194
195         text = build_yaml(args)
196         if args.output == "-":
197             sys.stdout.write(text)
198         else:
199             with open(args.output, "w", encoding="utf-8", newline="\n") as f:
200                 f.write(text)
201         return 0
202
203
204     if __name__ == "__main__":
205         raise SystemExit(main())
206
```

10. assistant (2026-07-07T14:46:41.268Z)

The README §2a is the key context ? it documents the exact deploy-coupling for #507's `--decode-backend`. Let me examine how the worker runs, how failure markers get written, and how args flow through the container entrypoint.

11. user (2026-07-07T14:46:42.922Z)

```
1       """Worker dispatcher ? pull from storage ? run the pipeline ? push outputs back.
2
3           python -m backend.worker infer --video-uri <uri> --out-uri <prefix> \
4               [--segmenter wasb_hybrid] [--config config.json] [--set k=v ...] [--scratch DIR]
5           python -m backend.worker train ...        # Gen-0 train-from-bucket (follow-up)
6
7       The URI scheme selects the storage backend via ``backend.storage`` (a bare path /
``local://``
8       is an identity passthrough; ``s3://`` / ``gcs://`` is an object store), so existing
local runs
9       are byte-for-byte unchanged. Combine with ``RALLY_DETECTOR_IMPL=stub`` and
10      ``--set indexing.skip_ai_handoff=true`` for a **secret-free, GPU-free** run (no Gemini
```

```
key) ?
    the path validated end-to-end on DigitalOcean (``docs/SETUP_NEW_MACHINE.md``).

    This is a thin orchestrator: it composes the SAME building blocks the ``backend.main``
CLI uses
    (validator registry ? segmenter ? report), never a forked pipeline.
    """

    from __future__ import annotations

    import argparse
    import csv
    import json
    import logging
    import os
    import re
    import sys
    import tempfile
    from typing import Any, List

    from backend import storage
    from backend.compute import ComputeJobSpec, get_compute_backend
    from backend.config import (
        StartupCheckError,
        enforce_startup_checks,
        load_config,
        probe_cuda_available,
    )
    from backend.infrastructure.database import Database
    from backend.pipeline.segmenters.base import segmenter_registry
    from backend.pipeline.validators.base import registry as validator_registry
    from backend.reporting import emit_indexer_report
    from backend.results import Failure
    from backend.utils.ffmpeg import ffprobe_bin  # P2a: single ffmpeg/ffprobe resolver
    from backend.utils.hashing import compute_video_id

    logger = logging.getLogger("backend.worker")


    def _coerce(v: str) -> Any:
        """Coerce a ``--set`` string value to bool/int/float where it round-trips, else
str."""
        low = v.lower()
        if low in ("true", "false"):
            return low == "true"
        for cast in (int, float):
            try:
                return cast(v)
            except ValueError:
                pass
        return v


    def _apply_overrides(config: Any, sets: List[str]) -> None:
        """Apply ``a.b.c=value`` overrides onto the typed config (e.g.
indexing.skip_ai_handoff=true)."""
        for kv in sets or []:
            key, sep, val = kv.partition("=")
            if not sep:
                raise ValueError(f"--set expects k=v, got {kv!r}")
            parts = key.split(".")
            obj = config
            for p in parts[:-1]:
                obj = getattr(obj, p)
            setattr(obj, parts[-1], _coerce(val))
```

```
72
73
74      def _write_intervals_csv(segments: List[dict], path: str) -> None:
75          """Decimal-second ``[start,end)`` rallies + shot count / ending reason ? the join-
friendly
76          artifact (same schema as the rally-annotator labels)."""
77          with open(path, "w", newline="") as f:
78              w = csv.writer(f)
79              w.writerow(["start", "end", "shot_count", "ending_reason"])
80              for s in segments:
81                  w.writerow(
82                      [
83                          f"{float(s.get('start_time', 0.0)):.3f}",
84                          f"{float(s.get('end_time', 0.0)):.3f}",
85                          s.get("shot_count", ""),
86                          s.get("ending_reason", s.get("shot_count_confidence", "")),
87                      ]
88                  )
89
90
91      def _write_report_json(
92          video_id: str, segments: List[dict], status: str, path: str, video_uri: str = ""
93      ) -> None:
94          with open(path, "w") as f:
95              json.dump(
96                  {
97                      "video_id": video_id,
98                      "status": status,
99                      "segment_count": len(segments),
100                     # The SOURCE the detection ran on (CP-rebuild P3, #485): a restart-wiped
101                     # control plane re-hydrating from outputs/<md5>/ restores the video row
102                     # with its truthful path ? without this, only the segments recover.
103                     "video_uri": video_uri,
104                     "segments": [
105                         {
106                             "start": float(s.get("start_time", 0.0)),
107                             "end": float(s.get("end_time", 0.0)),
108                         }
109                         for s in segments
110                     ],
111                 },
112                 f,
113                 indent=2,
114             )
115
116
117     def _serialize_and_push_outputs(
118         scratch: str,
119         out_uri: str,
120         video_id: str,
```

12. user (2026-07-07T14:46:43.731Z)

```
1       """``CloudRunCompute`` ? dispatch one inference job to a Cloud Run GPU **Job** and
bucket-poll
2       for its completion (COMPUTE_DECOUPLED_SERVING M6, decisions D16 + D7).
3
4       Flow (the cloud-serving compute path):
5         1. **Dispatch** a Cloud Run Job execution whose container runs ``python -m
backend.worker infer
6            --video-uri <gs> --out-uri <gs> --done-uri <gs> ?`` (the SAME worker, on the L4
GPU).
7         2. **Bucket-poll** ``done_uri`` via ``execution_policy.poll_until`` + the storage
layer (D16 ? the
8            completion signal is a tiny marker object the worker writes, NOT the Cloud Run API
```

```
status, so
9            it is storage-backed and reuses the existing park/resume primitives). Scale-to-zero
(D7): the
10           cold-start is absorbed by the async poll. On a poll TIMEOUT (``max_wait_sec`` can
undercut the
11           job's own ``--task-timeout`` budget) the shim does ONE final marker check ? a slow-
but-healthy
12           worker that finished just past the deadline is rescued, not wasted ? then best-
effort CANCELS
13           the still-running execution so the L4 doesn't keep billing until ``--task-
timeout``.
14        3. **Read** the marker (it carries ``video_id`` + the worker's returncode) ? fetch
15           ``outputs/<video_id>/report.json`` ? ``ComputeResult``.
16
17      The dispatched container is forced to run **INLINE** (``compute_target=local_gpu`` +
native detector)
18      so it never recursively re-dispatches ? the dispatcher decides cloud, the container just
computes.
19
20      The Cloud Run trigger is an **injected client** (``CloudRunJobClient``) so the whole
shim is unit-tested
21      with a fake (no GCP). The real ``GoogleCloudRunJobClient`` lazy-imports ``google-cloud-
run`` and is
22      owner-verified on the M7 real run.
23      """
24
25      from __future__ import annotations
26
27      import json
28      import logging
29      import os
30      import tempfile
31      import uuid
32      from abc import ABC, abstractmethod
33      from typing import Any, Callable, List, Optional
34
35      from backend.compute.base import ComputeBackend, ComputeJobSpec, ComputeResult
36      from backend.infrastructure.execution_policy import (
37          DEFAULT_POLICY,
38          ExecutionPolicy,
39          PolicyTimeout,
40          poll_until,
41      )
42      from backend.storage import fetch, get_storage, parse_uri
43
44      LOG = logging.getLogger("compute.cloud_run")
45
46      # The container worker must run INLINE on the GPU ? never re-enter the cloud-dispatch
branch.
47      # (compute_target=local_gpu disarms the cmd_infer dispatch guard; native = the L4 WASB
runner.)
48      _DEFAULT_CONTAINER_OVERRIDES: tuple[str, ...] = (
49          "deployment.compute_target=local_gpu",
50          "indexing.detector_impl=native",
51      )
52
53
54      class CloudRunJobClient(ABC):
55          """Triggers a Cloud Run **Job** execution with per-execution container arg
overrides.
56
57          Abstracted so ``CloudRunCompute`` is testable with a fake (no GCP). A real
implementation
58          overrides the job's container ``args`` for this execution and starts it."""
59
```

```
60          @abstractmethod
61          def run_job(
62              self,
63              job_name: str,
64              argv: List[str],
65              *,
66              region: str,
67              project: Optional[str] = None,
68          ) -> str:
69              """Start a Cloud Run Job execution running ``python -m backend.worker <argv>``;
return an
70              execution id/name (for logs). Does NOT block on completion (the caller bucket-
polls)."""
71
72          @abstractmethod
73          def cancel_execution(self, execution: str) -> None:
74              """Cancel a running execution by the name ``run_job`` returned. Called when the
bucket-poll
75              times out so the abandoned execution doesn't keep billing the GPU until
``--task-timeout``.
76              May raise ? the caller treats every failure as best-effort (the original poll
timeout is
77              NEVER masked by a cancel error)."""
78
79
80      class GoogleCloudRunJobClient(CloudRunJobClient):
81          """Real client ? lazy-imports ``google-cloud-run``. Owner-verified on the M7 real
run; CI uses
82          a fake, so this code path is never exercised in tests (the SDK isn't a test
dependency)."""
83
84          def run_job(
85              self,
86              job_name: str,
87              argv: List[str],
88              *,
89              region: str,
90              project: Optional[str] = None,
91          ) -> str:
92              # A None/empty project would build a bogus "projects/None/locations/..." path
(job_path
93              # requires a real str). Fail loudly BEFORE the SDK import so the misconfig is
exercisable
94              # in CI (where google-cloud-run is absent) and so mypy narrows project to str
for job_path.
95              if not project:
96                  raise RuntimeError(
97                      "GOOGLE_CLOUD_PROJECT is unset ? the real Cloud Run dispatch needs a GCP
project id "
98                      "to resolve the job path; set it on the control plane (see
get_compute_backend)."
99                  )
100             try:
101                 from google.cloud import run_v2  # type: ignore[attr-defined]
102             except Exception as exc:  # pragma: no cover - real SDK not present in CI
103                 raise RuntimeError(
104                     "google-cloud-run is required for CloudRunCompute's real client; install
it on the "
105                     "control plane (it is intentionally NOT a CI/test dependency)."
106                 ) from exc
107             client = (
108                 run_v2.JobsClient()
109             )  # pragma: no cover - exercised only on the real M7 run
110             name = client.job_path(project, region, job_name)
111             overrides = run_v2.RunJobRequest.Overrides(
```

```
                container_overrides=[
                    run_v2.RunJobRequest.Overrides.ContainerOverride(args=argv)
                ]
            )
            op = client.run_job(
                request=run_v2.RunJobRequest(name=name, overrides=overrides)
            )
            return (
                getattr(op, "metadata", None)
                and getattr(op.metadata, "name", "")
                or "execution"
            )

        def cancel_execution(self, execution: str) -> None:
            # run_job falls back to the literal string "execution" when the operation
metadata carries
            # no name ? that is NOT a resolvable resource name. Fail loudly BEFORE the SDK
import
            # (mirrors run_job's project guard, #342) so the misuse is exercisable in CI
(where
            # google-cloud-run is absent).
            if "/executions/" not in (execution or ""):
                raise RuntimeError(
                    f"not a fully-qualified Cloud Run execution name: {execution!r} ?
expected "
                    "projects/<p>/locations/<l>/jobs/<j>/executions/<e> (run_job could not
resolve "
                    "one from the operation metadata, so there is nothing to cancel by
name)."
                )
            try:
                from google.cloud import run_v2  # type: ignore[attr-defined]
            except Exception as exc:  # pragma: no cover - real SDK not present in CI
                raise RuntimeError(
                    "google-cloud-run is required to cancel a Cloud Run execution; install
it on the "
                    "control plane (it is intentionally NOT a CI/test dependency)."
                ) from exc
            client = (
                run_v2.ExecutionsClient()
            )  # pragma: no cover - exercised only on a real run
            client.cancel_execution(request=run_v2.CancelExecutionRequest(name=execution))


    class CloudRunCompute(ComputeBackend):
        """Dispatch to a Cloud Run GPU Job + bucket-poll for completion."""

        def __init__(
            self,
            *,
            run_client: CloudRunJobClient,
            job_name: str,
            region: str,
            project: Optional[str] = None,
            storage_config: Optional[dict] = None,
            policy: ExecutionPolicy = DEFAULT_POLICY,
            token: Optional[str] = None,
            container_overrides: Optional[tuple[str, ...]] = None,
        ) -> None:
            self._client = run_client
            self._job_name = job_name
            self._region = region
            self._project = project
            self._storage_config = storage_config or {}
            self._policy = policy
```

```
            # The dispatch token keys the done-marker so concurrent jobs to one bucket don't
collide.
            # None ? a fresh uuid4 per run_infer call (prod); injected ? deterministic
(tests).
            self._token = token
            self._container_overrides = (
                _DEFAULT_CONTAINER_OVERRIDES
                if container_overrides is None
                else tuple(container_overrides)
            )

        def run_infer(
            self,
            spec: ComputeJobSpec,
            *,
            on_wait: "Callable[[float], None] | None" = None,
        ) -> ComputeResult:
            out_root = spec.out_uri.rstrip("/")
            token = self._token or uuid.uuid4().hex
            done_uri = f"{out_root}/_dispatch/{token}.done"
            argv: List[str] = [
                "infer",
                "--video-uri",
                spec.video_uri,
                "--out-uri",
                spec.out_uri,
                "--done-uri",
                done_uri,
            ]
            if spec.segmenter:
                argv += ["--segmenter", spec.segmenter]
            if spec.skip_validation:
                # The control-plane already validated + only dispatches on PASS ? tell the
worker not to
                # re-decode and re-validate (redundant, and its config may resolve different
thresholds).
                argv.append("--skip-validation")
            for kv in (*spec.config_overrides, *self._container_overrides):
                argv += ["--set", kv]

            execution = self._client.run_job(
                self._job_name, argv, region=self._region, project=self._project
            )
            LOG.info(
                "cloud-run: dispatched job=%s exec=%s; bucket-polling %s",
                self._job_name,
                execution,
                done_uri,
            )

            try:
                marker = poll_until(
                    lambda: self._read_marker(done_uri),
                    self._policy,
                    label="cloud-run-infer",
                    logger=LOG,
                    # Live heartbeat (#482): each still-waiting tick reaches the caller so
                    # job.progress moves during the whole GPU run instead of freezing.
                    on_tick=on_wait,
                )
            except PolicyTimeout as timeout_exc:
                marker = self._handle_poll_timeout(done_uri, str(execution), timeout_exc)
            video_id = str(marker.get("video_id", ""))
            # A MISSING returncode defaults to FAILURE (1), never success ? a present-but-
unreadable /
```

```
            # garbled marker must not masquerade as rc=0. (The worker always writes both
fields.)
            rc = int(marker.get("returncode", 1))
            if not video_id:
                LOG.warning(
                    "cloud-run: job=%s completion marker present but empty/unreadable (no "
                    "video_id) ? treating as failure",
                    self._job_name,
                )
                rc = rc or 1
            outputs_uri = f"{out_root}/outputs/{video_id}/" if video_id else ""
            report = self._read_json(f"{outputs_uri}report.json") if outputs_uri else {}
            LOG.info(
                "cloud-run: job=%s done (video_id=%s rc=%d)", self._job_name, video_id, rc
            )
            return ComputeResult(
                returncode=rc,
                outputs_uri=outputs_uri,
                video_id=video_id,
                report=report,
                execution=str(execution or ""),
            )

        def _handle_poll_timeout(
            self, done_uri: str, execution: str, timeout_exc: PolicyTimeout
        ) -> dict:
            """Poll deadline hit ? rescue a just-finished result, else cancel + re-raise.

            The poll budget (``max_wait_sec``, default 1800s) can undercut the deployed
job's own
            ``--task-timeout`` (3600s), so a slow-but-healthy worker may finish just past
the deadline:
            ONE final marker check rescues that completed GPU run instead of wasting it. If
the marker
            is genuinely absent, the execution is still running ? best-effort cancel it
(otherwise the
            L4 keeps billing until ``--task-timeout``) and re-raise a ``PolicyTimeout``
carrying the
            execution name so the operator can verify the state in the GCP console. A cancel
failure
            NEVER masks the original timeout."""
            try:
                late = self._read_marker(done_uri)
            except Exception:  # noqa: BLE001 ? the final check is best-effort too: absence
? failure
                LOG.warning(
                    "cloud-run: final post-timeout marker check failed ? treating as
absent",
                    exc_info=True,
                )
                late = None
            if late is not None:
                LOG.warning(
                    "cloud-run: job=%s exec=%s done-marker found on the final post-timeout
check "
                    "? rescuing the completed result",
                    self._job_name,
                    execution,
                )
                return late
            try:
                self._client.cancel_execution(execution)
            except Exception as cancel_exc:  # noqa: BLE001 ? best-effort: never mask the
timeout
                cancel_note = f"best-effort cancel FAILED ({cancel_exc})"
```

```
284                 LOG.warning(
285                     "cloud-run: best-effort cancel of execution=%s failed: %s",
286                     execution,
287                     cancel_exc,
288                     exc_info=True,
289                 )
290             else:
291                 cancel_note = "cancel requested"
292                 LOG.info("cloud-run: requested cancel of execution=%s", execution)
293             raise PolicyTimeout(
294                 f"{timeout_exc} [cloud-run job={self._job_name} "
295                 f"execution={execution or '<unknown>'}; {cancel_note} ? verify the execution
"
296                 "state in the GCP console]"
297             ) from timeout_exc
298 
299         # --- storage-backed completion signal (bucket-poll) ---
300         def _read_marker(self, done_uri: str) -> Optional[dict]:
301             """The done-marker dict if the worker has written it, else None ? poll_until
keeps waiting."""
302             ref = parse_uri(done_uri)
303             if not get_storage(ref.backend, config=self._storage_config).exists(ref):
304                 return None
305             return (
306                 self._read_json(done_uri) or {}
307             )  # empty dict still terminates the poll (present == done)
308 
309         def _read_json(self, uri: str) -> dict:
310             # Download to a real local dst FIRST: the cloud backends (gcs/s3) REQUIRE a dst
? a None dst
311             # raises ValueError ? unlike local's identity passthrough. Without an explicit
dst this read
312             # would silently return {} for every gs://|s3:// object (the only backends this
shim targets).
313             try:
314                 ref = parse_uri(uri)
315                 with tempfile.TemporaryDirectory(prefix="cr-read-") as td:
316                     dst = os.path.join(td, ref.name or "obj.json")
317                     local = fetch(uri, dst, config=self._storage_config)
318                     with open(local, "r", encoding="utf-8") as f:
319                         data: Any = json.load(f)
320                 return data if isinstance(data, dict) else {}
321             except Exception:
322                 LOG.warning("cloud-run: could not read %s", uri, exc_info=True)
323                 return {}
324 
```

13. assistant (2026-07-07T14:46:50.709Z)

Now let me examine the worker's main(), how the done-marker/failure-marker gets written, and the `--done-uri` handling.

14. user (2026-07-07T14:46:52.220Z)

```
240         # M6: a cloud compute_target DISPATCHES to a Cloud Run Job instead of running in-
process.
241         # DEFAULT-OFF: get_compute_backend returns None for ""/local_gpu/cpu_mock ? fall
through to the
242         # unchanged inline path below. The dispatched container runs THIS cmd_infer inline
(the
243         # dispatcher forces compute_target=local_gpu), so it never recursively re-
dispatches.
244         compute = get_compute_backend(config)
245         if compute is not None:
246             return _dispatch_remote(compute, args)
```

```
247
248         storage_cfg = config.storage.model_dump()
249
250         scratch = args.scratch or tempfile.mkdtemp(prefix="rally-worker-")
251         os.makedirs(scratch, exist_ok=True)
252         config.output.output_dir = scratch
253
254         # 1. PULL ? local:// / bare path = identity passthrough; s3:// / gcs:// = download
to scratch.
255         local_video = storage.fetch(
256             args.video_uri, os.path.join(scratch, "in.mp4"), config=storage_cfg
257         )
258         print(f"[worker] pulled {args.video_uri} -> {local_video}")
259
260         # 2. IDENTITY ? whole-file MD5 (the labels?video join key; pulled bytes must be
byte-identical).
261         video_id = compute_video_id(local_video)
262
263         # 3. VALIDATE (same registry as the main CLI) ? UNLESS the dispatcher already did.
264         # On the cloud-serving path the CONTROL-PLANE is the authoritative validation gate:
it runs the
265         # SAME validators with ITS resolved config and only dispatches on PASS. Re-
validating here is
266         # redundant, decodes the whole video a SECOND time, and is outright WRONG when the
worker's config
267         # resolves different thresholds than the control-plane's ? observed 2026-07-07: a
5312x2988 clip
268         # (sharpness 11.9) PASSED the control-plane at validation.min_blur_threshold=8.0,
then the worker
269         # REJECTED it at its own default 20.0, so a SUCCESSFUL GPU dispatch failed after the
fact.
270         # --skip-validation (set by the dispatcher in orchestration._maybe_dispatch_remote)
skips it; the
271         # direct ``worker infer`` CLI has no control-plane, so it still validates by
default.
272         if getattr(args, "skip_validation", False):
273             print(
274                 "[worker] validation SKIPPED (--skip-validation): the dispatcher is the
authoritative "
275                 "gate and already validated this input."
276             )
277             val_results, val_context = [], {}
278         else:
279             val_results, val_context = validator_registry.run_all_with_context(
280                 local_video, config
281             )
282         failures = [r for r in val_results if not r.passed]
283         db = Database(os.path.join(scratch, config.output.db_name))
284         db.add_video(
285             video_id,
286             local_video,
287             "failed" if failures else "processed",
288             [r.model_dump() for r in val_results],
289         )
290
291         def _fail(rc: int) -> int:
292             # A worker-side FAILURE (validation, unknown segmenter, or a detector
timeout/abort the
293             # segmenter surfaced as a Failure) must NOT masquerade as a 0-rally "success".
Write + push
294             # a status="failed" report.json/intervals.csv (0 segments) so the control
plane's bucket
295             # truth (submit-cache probe, restart-recovery) and the alpha-user result surface
see an
296             # explicit FAILURE ? the wf294 silent-failure fix. Best-effort: a push hiccup
```

```
must never
297             # mask the original failure. THEN the M6 done-marker (real rc + status) so a
remote
298             # dispatcher's bucket-poll terminates FAST instead of hanging until its budget
expires.
299             try:
300                 _serialize_and_push_outputs(
301                     scratch,
302                     args.out_uri,
303                     video_id,
304                     [],
305                     "failed",
306                     storage_cfg,
307                     str(getattr(args, "video_uri", "") or ""),
308                 )
309             except Exception as e:  # never mask the real failure on an artifact-push hiccup
310                 logger.warning("[worker] could not push failure artifacts: %s", e)
311             # DEFAULT-OFF: no --done-uri ? no marker written (the unchanged local run).
312             if getattr(args, "done_uri", None):
313                 _write_done_marker(
314                     args.done_uri, video_id, rc, 0, "failed", storage_cfg, scratch
315                 )
316             return rc
317
318         segments: List[dict] = []
319         segmenter_actual = None
320         segmentation_ran = False
321         status = "failed"
322         try:
323             if failures:
324                 print(
325                     "[worker] validation FAILED: "
326                     + "; ".join(f"{r.validator_name}: {r.message}" for r in failures)
327                 )
328                 return _fail(2)
329             seg_name = args.segmenter or config.indexing.default_segmenter
330             segmenter_cls = segmenter_registry.get(seg_name)
331             if not segmenter_cls:
332                 print(
333                     f"[worker] unknown segmenter: {seg_name!r} "
334                     f"(have {list(segmenter_registry.list_available())})"
335                 )
336                 return _fail(2)
337             segmenter_actual = seg_name
338             result = segmenter_cls(db, config).process_video(
339                 video_id, local_video, max_frames=args.max_frames
340             )
341             segmentation_ran = True
342             if isinstance(result, Failure):
343                 print(f"[worker] segmenter FAILED: {result.message}")
344                 return _fail(3)
345             segments = result.value if hasattr(result, "value") else result
346             status = "success"
347             # Loud, never-silent diagnostic: a 0-segment success is almost always a
misconfig, not an
348             # empty match ? the cold-start failure mode (the substrate detector yielded no
usable
349             # trajectories). Explain it so a run is analysable from the log alone (re-run -v
for more).
350             if not segments:
351                 detector_impl = (
352                     os.environ.get("RALLY_DETECTOR_IMPL")
353                     or getattr(config.indexing, "detector_impl", None)
354                     or "auto"
355                 )
```

```
356                 logger.warning(
357                     "infer produced 0 segments for video_id=%s (segmenter=%s,
detector_impl=%s). "
358                     "The detector substrate yielded no usable trajectories/windows ? check
the detector "
359                     "logs above; common causes: a missing ${AI_PROVIDER}_API_KEY with "
```

15. user (2026-07-07T14:46:53.122Z)

```
640                     }
641                 )
642
643         if len(specs) < 2:
644             print(
645                 f"[worker] need >=2 videos with trajectories for leave-one-video-out; got
{len(specs)}."
646             )
647             return 2
648         manifest_path = os.path.join(scratch, "manifest.json")
649         with open(manifest_path, "w", encoding="utf-8") as f:
650             json.dump(specs, f, indent=2)
651         src_label = (
652             "real detector" if args.trajectory_source == "detector" else "CPU-mock stub"
653         )
654         print(f"[worker] built manifest: {len(specs)} videos ({src_label} trajectories)")
655
656         out_dir = os.path.join(scratch, "artifacts")
657         cmd = [
658             sys.executable,
659             "-m",
660             "training.gen0.harness",
661             "--manifest",
662             manifest_path,
663             "--out",
664             out_dir,
665             "--version",
666             args.version,
667         ]
668         if args.floor:
669             floor_local = (
670                 storage.fetch(
671                     args.floor, os.path.join(scratch, "floor.json"), config=storage_cfg
672                 )
673                 if "://" in args.floor
674                 else args.floor
675             )
676             cmd += ["--floor", floor_local]
677         print("[worker] training:", " ".join(cmd))
678         rc = subprocess.run(cmd).returncode
679         if rc != 0:
680             print(f"[worker] gen0 harness failed (rc={rc})")
681             return rc
682
683         # PUSH the versioned artifact bundle (weights.json + manifest.json) to the bucket.
684         bundle = os.path.join(out_dir, args.version)
685         out_prefix = args.out_uri.rstrip("/")
686         pushed = []
687         for fn in sorted(os.listdir(bundle)):
688             uri = f"{out_prefix}/weights/{args.version}/{fn}"
689             storage.put(os.path.join(bundle, fn), uri, config=storage_cfg)
690             pushed.append(uri)
691         print(f"[worker] train done: pushed {len(pushed)} artifact(s):")
692         for u in pushed:
693             print("   ", u)
694         return 0
```

```
695
696
697     def build_parser() -> argparse.ArgumentParser:
698         p = argparse.ArgumentParser(
699             prog="python -m backend.worker",
700             description="Storage-aware worker: pull ? run pipeline ? push.",
701         )
702         p.add_argument(
703             "-v",
704             "--verbose",
705             action="store_true",
706             help="DEBUG logging (the native detector command, frame counts, per-stage
detail)",
707         )
708         sub = p.add_subparsers(dest="cmd", required=True)
709
710         pi = sub.add_parser("infer", help="run inference on one video URI and push outputs")
711         pi.add_argument(
712             "--video-uri", required=True, help="local path / local:// / s3:// / gcs://"
713         )
714         pi.add_argument(
715             "--out-uri",
716             required=True,
717             help="output prefix (outputs/<video_id>/? is appended)",
718         )
719         pi.add_argument(
720             "--segmenter", default=None, help="default: indexing.default_segmenter"
721         )
722         pi.add_argument(
723             "--config", default=None, help="config.json path (default: ./config.json)"
724         )
725         pi.add_argument(
726             "--scratch", default=None, help="local scratch dir (default: a temp dir)"
727         )
728         pi.add_argument("--max-frames", type=int, default=None)
729         pi.add_argument(
730             "--done-uri",
731             default=None,
732             help="M6: storage URI for a completion MARKER (video_id + rc) written on a "
733             "successful run, so a remote (Cloud Run) dispatcher's bucket-poll terminates. "
734             "Default-OFF: omit it for the normal local run.",
735         )
736         pi.add_argument(
737             "--set",
738             action="append",
739             default=[],
740             metavar="k=v",
741             help="config override, e.g. --set indexing.skip_ai_handoff=true",
742         )
743         pi.add_argument(
744             "--skip-validation",
745             action="store_true",
746             help="skip the worker's re-validation: the dispatcher (control-plane) already
ran the "
747             "validators with its resolved config and only dispatches on PASS, so re-
validating here is "
748             "redundant, decodes the video again, and can contradict the gate. Default-OFF:
the direct "
749             "`worker infer` CLI still validates.",
750         )
751         pi.set_defaults(func=cmd_infer)
752
753         pt = sub.add_parser(
754             "train",
755             help="Gen-0 train-from-bucket (labels [+videos] -> trajectories -> harness)",
```

```
756         )
757         pt.add_argument(
758             "--corpus-uri",
759             required=True,
760             help="prefix containing labels/ (+ videos/ for "
761             "--trajectory-source detector), e.g. s3://bucket or gcs://bucket",
762         )
763         pt.add_argument(
764             "--out-uri",
765             required=True,
766             help="output prefix (weights/<version>/ is appended)",
767         )
768         pt.add_argument("--version", required=True, help="semver for the artifact bundle")
769         pt.add_argument(
770             "--trajectory-source",
771             choices=["synth", "detector"],
772             default="synth",
773             help="synth = CPU-mock label-consistent trajectories (default, no GPU); "
774             "detector = REAL shuttle trajectories via the resolved DetectorRunner "
775             "(needs videos/ in the corpus; honours detector_impl).",
776         )
777         pt.add_argument(
778             "--segmenter",
779             default="wasb_hybrid",
780             help="trajectory hybrid whose detector is used for --trajectory-source
detector",
781         )
782         pt.add_argument(
783             "--floor", default=None, help="floor baseline JSON (local path or storage URI)"
784         )
785         pt.add_argument("--config", default=None)
786         pt.add_argument("--scratch", default=None)
787         pt.add_argument(
788             "--fps",
789             type=float,
790             default=30.0,
791             help="synth trajectory fps (detector probes the video)",
792         )
793         pt.add_argument(
794             "--frame-width",
795             type=float,
796             default=1280.0,
797             help="synth trajectory frame width (detector probes the video)",
798         )
799         pt.add_argument("--set", action="append", default=[], metavar="k=v")
```

16. assistant (2026-07-07T14:47:15.122Z)

Let me examine how #509 forwards decode_backend (via `--set` vs a dedicated CLI arg) and read the full `_maybe_dispatch_remote`, plus how the failure marker relates to argparse exit.

17. user (2026-07-07T14:47:16.752Z)

No matches found

18. user (2026-07-07T14:47:17.941Z)

```
520                     f"cloud-serving: failed to stage the uploaded file to {input_root}:
{e}",
521                     False,
522                 )
523             # Repoint BOTH the dispatch and the DB video record at the staged bucket object:
the L4 job
524             # DETECTs from it, and a later /api/compile re-fetches the same source (the
```

```
local upload is
525             # gone once this instance recycles). add_video is an UPSERT ? it only updates
filepath.
526             req.video_path = staged_uri
527             input_ref = storage.resolve_input_ref(staged_uri, storage_cfg)
528             db.add_video(
529                 video_id, staged_uri, "processed", [r.model_dump() for r in val_results]
530             )
531         out_root = (getattr(storage_cfg, "output_root", "") or "").strip()
532         if not out_root:
533             return _failed(
534                 "cloud-serving needs storage.output_root set to a bucket (e.g. gs://your-
bucket) "
535                 "? that's where the Cloud Run job writes the result.",
536                 False,
537             )
538
539         notify("dispatching (cloud_run)")
540         logger.info(
541             "[API] cloud-serving: dispatching %s -> Cloud Run Job (out=%s)",
542             req.video_path,
543             out_root,
544         )
545         # Forward the engine's AI-handoff stance so the cloud job runs the SAME detection:
with
546         # skip_ai_handoff=True the WASB windows become rallies directly (no Gemini key
needed); without
547         # forwarding the cloud job would default to the handoff and yield 0 rallies when it
has no key.
548         # (player_pool / max_frames are still dropped on the cloud path ? a separate
documented follow-up.)
549         overrides = []
550         idx_cfg = getattr(cfg, "indexing", None)
551         sk = getattr(idx_cfg, "skip_ai_handoff", None)
552         if sk is not None:
553             overrides.append(f"indexing.skip_ai_handoff={'true' if sk else 'false'}")
554         # Forward the resolved WASB serving knobs so the PRESET governs the worker
(#506/#507). The
555         # worker Job has no DEPLOYMENT_PROFILE, so it does NOT apply the cloud-serving
preset itself ?
556         # without forwarding, its decode_backend + GPU-feed tuning silently fall back to the
model
557         # defaults (cpu, batch 8) regardless of what the control-plane resolved.
decode_backend is
558         # cuda-only (validator-enforced) and requires a worker image that understands
--decode-backend
559         # (? #507); batch_size/prefetch_batches are forwarded only when set (None = the
parity default).
560         wasb_cfg = getattr(idx_cfg, "wasb", None)
561         if wasb_cfg is not None:
562             decode_backend = getattr(wasb_cfg, "decode_backend", None)
563             if decode_backend:
564                 overrides.append(f"indexing.wasb.decode_backend={decode_backend}")
565             batch_size = getattr(wasb_cfg, "batch_size", None)
566             if batch_size:
567                 overrides.append(f"indexing.wasb.batch_size={batch_size}")
568             prefetch = getattr(wasb_cfg, "prefetch_batches", None)
569             if prefetch:
570                 overrides.append(f"indexing.wasb.prefetch_batches={prefetch}")
571         # Tell the worker to SKIP its redundant re-validation: THIS control-plane is the
authoritative
572         # gate ? it already ran the validators with its resolved config just above (only a
PASS reaches
573         # here; a failure returned "failed" before dispatch). A worker re-run would decode
the video a
```

```
574         # second time and, because its config isn't fully forwarded, can resolve DIFFERENT
thresholds and
575         # REJECT a clip this gate already passed (observed 2026-07-07: passed at
min_blur_threshold=8.0,
576         # then worker-rejected at its default 20.0 after a successful GPU dispatch).
577         spec = ComputeJobSpec(
578             video_uri=req.video_path,
579             out_uri=out_root,
580             segmenter=seg_name,
581             config_overrides=tuple(overrides),
582             skip_validation=True,
583         )
584         try:
585             # Dispatch the Cloud Run Job + bucket-poll to completion. on_wait is the live
586             # heartbeat (#482): job.progress moves on every poll tick instead of freezing at
587             # "segmenting" for the whole GPU run (the 2026-07-05 silent 22-min spinner) ?
which
588             # also gives the SPA a stall signal to time out on, instead of a blind wall
clock.
589             result = compute.run_infer(
590                 spec,
591                 on_wait=lambda elapsed: notify(_wait_progress_message(seg_name, elapsed)),
592             )
593         except Exception as e:  # noqa: BLE001 ? any dispatch/poll error is a clean job
failure, not a 500
594             logger.error("[API] cloud dispatch error: %s", e, exc_info=True)
595             return _failed(f"cloud dispatch error: {e}", True)
596         if result.returncode != 0:
597             return _failed(f"cloud job failed (returncode {result.returncode}).", True)
598         if result.video_id and result.video_id != video_id:
599             # The worker re-hashes the bucket bytes; a divergence means the input changed
between this
600             # box's hash and the worker's ? ingest under the LOCAL id but flag it so it's
diagnosable.
601             logger.warning(
602                 "[API] cloud video_id %s != local %s ? input bytes differ between the API
hash "
603                 "and the worker hash; ingesting under the local id.",
604                 result.video_id,
605                 video_id,
606             )
607
608         notify("ingesting (cloud_run)")
609         storage_dict = (
610             storage_cfg.model_dump()  # type: ignore[union-attr]  # hasattr-guarded; mypy
can't narrow
611             if hasattr(storage_cfg, "model_dump")
612             else {}
613         )
614         try:
615             _ingest_remote_segments(db, video_id, result.outputs_uri, storage_dict)
616         except Exception as e:  # noqa: BLE001 ? a missing/partial/corrupt result must not
500 or leak rows
617             logger.error("[API] cloud ingest failed: %s", e, exc_info=True)
618             return _failed(
619                 f"cloud-serving: could not read the result from {result.outputs_uri}: {e}",
620                 True,
621             )
622
623         # Only the NEW cloud rows (id > the pre-run watermark); _finalize_reingest then
drops the old ones.
624         segments = [
625             s for s in db.query_play_segments(video_id, {}) if int(s.get("id", 0)) > play_wm
626         ]
627         _finalize_reingest(db, video_id, segments, play_wm, cand_wm)
```

```
628         # Compute-path PROVENANCE: the GCP-verifiable proof the cloud path ran. `execution`
is the real
629         # Cloud Run execution name (cross-check: gcloud run jobs executions describe
<execution>).
630         provenance = {
631             "path": "cloud_run",
632             "requested": req.compute,
633             "job": os.environ.get("CLOUD_RUN_JOB", "rally-worker"),
634             "region": os.environ.get("CLOUD_RUN_REGION", "asia-south1"),
635             "execution": result.execution,
636             "outputs_uri": result.outputs_uri,
637         }
638         logger.info(
639             "[COMPUTE] cloud_run dispatched: execution=%s job=%s region=%s out=%s
video_id=%s",
```

19. assistant (2026-07-07T14:48:04.100Z)

This confirms the key asymmetry. Let me verify the argparse flow in `main()` and confirm the done-marker is written only after parse succeeds, plus check the `ComputeResult`/rc-4 mapping and the frozen-dataclass concern.

20. user (2026-07-07T14:48:04.271Z)

```
800         pt.set_defaults(func=cmd_train)
801         return p
802
803
804     def _setup_logging(verbose: bool) -> None:
805         """Configure logging for the worker CLI so the pipeline's existing INFO/ERROR
diagnostics are
806         VISIBLE ? the worker previously ran with no logging config, so every
segmenter/detector log
807         (detector choice, 'Running native WASB inference', availability failures) was
silently dropped.
808         That is exactly why a 0-rally cold-start gave nothing to analyse. Configured here
(the CLI
809         entry), not at import, so importing the module never reconfigures a caller's
logging."""
810         # force=True: basicConfig is a no-op if the root already has handlers (e.g. a
library
811         # configured logging at import), which would leave the level at WARNING and re-
suppress the
812         # diagnostics. Forcing reconfiguration means the worker reliably controls its own
verbosity.
813         logging.basicConfig(
814             level=logging.DEBUG if verbose else logging.INFO,
815             format="%(asctime)s [%(levelname)s] %(name)s: %(message)s",
816             datefmt="%H:%M:%S",
817             force=True,
818         )
819
820
821     def main(argv=None) -> int:
822         args = build_parser().parse_args(argv)
823         _setup_logging(getattr(args, "verbose", False))
824         return args.func(args)
825
826
827     if __name__ == "__main__":
828         sys.exit(main())
829
```

21. user (2026-07-07T14:48:04.751Z)

```
215     ) -> None:
216         """M6: write a tiny JSON completion marker to ``done_uri`` (via the storage layer)
for a remote
217         dispatcher's bucket-poll. Best-effort ? never raises into the worker's success
return."""
218         try:
219             marker = {
220                 "video_id": video_id,
221                 "returncode": returncode,
222                 "segment_count": segment_count,
223                 "status": status,
224             }
225             local = os.path.join(scratch, "_done.json")
226             with open(local, "w", encoding="utf-8") as f:
227                 json.dump(marker, f)
228             storage.put(local, done_uri, config=storage_cfg)
229             logger.info("[worker] wrote completion marker -> %s", done_uri)
230         except Exception as e:  # never fail a good run because the marker push hiccuped
231             logger.warning("[worker] could not write done-marker %s: %s", done_uri, e)
232
233
234     def cmd_infer(args: argparse.Namespace) -> int:
235         config = load_config(args.config) if args.config else load_config()
236         _apply_overrides(config, args.set)
237         if not _run_startup_checks(config):
238             return 2
239
240         # M6: a cloud compute_target DISPATCHES to a Cloud Run Job instead of running in-
process.
241         # DEFAULT-OFF: get_compute_backend returns None for ""/local_gpu/cpu_mock ? fall
through to the
242         # unchanged inline path below. The dispatched container runs THIS cmd_infer inline
(the
243         # dispatcher forces compute_target=local_gpu), so it never recursively re-
dispatches.
244         compute = get_compute_backend(config)
245         if compute is not None:
246             return _dispatch_remote(compute, args)
247
248         storage_cfg = config.storage.model_dump()
249
250         scratch = args.scratch or tempfile.mkdtemp(prefix="rally-worker-")
251         os.makedirs(scratch, exist_ok=True)
252         config.output.output_dir = scratch
253
254         # 1. PULL ? local:// / bare path = identity passthrough; s3:// / gcs:// = download
to scratch.
```

22. user (2026-07-07T14:48:05.410Z)

```
1       """The ``ComputeBackend`` ABC ? one interface for *where* the rally core runs.
2
3       COMPUTE_DECOUPLED_SERVING M6. Mirrors the ``StorageBackend`` ABC+factory idiom
4       (``backend/storage/``): a ``ComputeBackend`` decides **where** the pure
DETECT?WINDOW?STITCH
5       core executes ? in-process on the local box (truly-local) or on a **Cloud Run GPU Job**
6       (cloud-serving) ? without the core ever branching on the profile (D1).
7
8       The seam is **default-OFF**: only ``deployment.compute_target == "cloud_run"`` selects a
9       non-local backend. Every other target (``""`` / ``local_gpu`` / ``cpu_mock``) keeps the
worker's
10      existing in-process path **byte-identical** ? the factory returns ``None`` (= "run
inline").
11      """
12
```

```
    from __future__ import annotations

    from abc import ABC, abstractmethod
    from dataclasses import dataclass, field
    from typing import Callable, Optional


    @dataclass(frozen=True)
    class ComputeJobSpec:
        """One inference job, profile-agnostic. ``video_id`` is the MD5 the worker will (re)compute
        from the SAME input bytes, so the output always lands at ``<out_uri>/outputs/<video_id>/`` ?
        deterministic regardless of where the job ran."""

        video_uri: str  # input (gs://? for cloud; a path/local:// for local)
        out_uri: str  # output ROOT (gs://? bucket for cloud)
        segmenter: Optional[str] = None
        # Raw worker ``--set k=v`` overrides. A tuple (not list/dict) so the spec stays frozen/hashable.
        config_overrides: tuple[str, ...] = ()
        # Skip the worker's redundant re-validation. The DISPATCHER (the cloud-serving control-plane) is
        # the authoritative validation gate: it runs the SAME validators with ITS resolved config and only
        # dispatches on PASS, so a worker re-run is (a) redundant, (b) a second full-video decode, and
        # (c) outright WRONG when the worker's config resolves different thresholds ? it can REJECT a clip
        # the gate already passed. Default False keeps the direct ``worker infer`` CLI (no control-plane)
        # validating exactly as before.
        skip_validation: bool = False


    @dataclass(frozen=True)
    class ComputeResult:
        """The terminal state of a dispatched job. ``returncode`` mirrors the worker's exit code
        (0 ok / 2 validation-or-config / 3 segmenter-failure), plus **4 = remote dispatch timeout or an
        empty/unreadable completion marker** (the cloud path only). ``video_id`` is known once the job's
        done-marker is read; ``outputs_uri`` is where ``intervals.csv`` / ``report.json`` landed."""

        returncode: int
        outputs_uri: str = ""
        video_id: str = ""
        report: dict = field(default_factory=dict)
        # The backend's execution identity for PROVENANCE (e.g. the Cloud Run execution name) ? surfaced
        # on the job result so "the cloud path actually ran" is independently verifiable (gcloud run jobs
        # executions describe <execution>). Empty for the in-process path / backends without one.
        execution: str = ""


    class ComputeBackend(ABC):
        """Backend-agnostic compute. ``run_infer`` dispatches ONE inference job and returns only once
        its outputs are durably available (bucket-poll for cloud), so a caller treats local and cloud
        execution identically."""
```

```
61
62          @abstractmethod
63          def run_infer(
64              self,
65              spec: ComputeJobSpec,
66              *,
67              on_wait: "Callable[[float], None] | None" = None,
68          ) -> ComputeResult:
69              """Run ``spec`` to completion and return its ``ComputeResult``.
70
71              ``on_wait(elapsed_sec)`` (optional) is invoked periodically while the backend is
72              still WAITING on the remote work ? the caller's live-progress heartbeat (#482),
so a
73              long GPU run doesn't look frozen. Implementations without a wait loop may ignore
it."""
74
```

23. assistant (2026-07-07T14:48:25.399Z)

Let me confirm the poll timeout duration (to quantify the hang) and check for any positional `ComputeJobSpec` constructions that the new field could break.

24. user (2026-07-07T14:48:27.035Z)

No matches found

25. user (2026-07-07T14:48:27.515Z)

No matches found

26. user (2026-07-07T14:48:34.596Z)

```
backend/orchestration.py:577:    spec = ComputeJobSpec(
backend/worker/__main__.py:184:    spec = ComputeJobSpec(
tests/test_compute_backend.py:184:        ComputeJobSpec(
tests/test_compute_backend.py:209:        ComputeJobSpec(
tests/test_compute_backend.py:237:        ComputeJobSpec(video_uri="gs://b/v.mp4",
out_uri=str(tmp_path / "bk"))
tests/test_compute_backend.py:240:        ComputeJobSpec(video_uri="gs://b/v.mp4",
out_uri=str(tmp_path / "bk"))
tests/test_compute_backend.py:308:        ComputeJobSpec(video_uri="gs://b/v.mp4",
out_uri="gs://b")
tests/test_compute_backend.py:335:            ComputeJobSpec(video_uri="gs://b/v.mp4",
out_uri=str(tmp_path / "bucket"))
tests/test_compute_backend.py:357:            ComputeJobSpec(video_uri="gs://b/v.mp4",
out_uri=str(tmp_path / "bucket"))
tests/test_compute_backend.py:394:        ComputeJobSpec(video_uri="gs://b/v.mp4",
out_uri="gs://b")
tests/test_compute_backend.py:780:        ComputeJobSpec(video_uri="gs://b/v.mp4",
out_uri=str(tmp_path / "bk")),
```

27. user (2026-07-07T14:48:35.397Z)

```
8:  ``max_wait_sec`` deadline).
42:    """An op hung past ``op_timeout_sec`` or never became ready within ``max_wait_sec``."""
54:    max_wait_sec: float = 1800.0  # total wall budget for a poll loop
75:DEFAULT_POLICY = ExecutionPolicy()
85:    policy: ExecutionPolicy = DEFAULT_POLICY,
154:    policy: ExecutionPolicy = DEFAULT_POLICY,
163:    ``PolicyTimeout`` after ``max_wait_sec``. ``check`` returns ``None`` while the op is
still in progress.
177:        policy.max_wait_sec,
187:        if elapsed >= policy.max_wait_sec:
192:                f"{label} not ready within max_wait_sec={policy.max_wait_sec}s"
```

28. assistant (2026-07-07T14:48:57.362Z)

Let me confirm the worker Job's container entrypoint (Dockerfile) to be certain the argparse abort path is what runs, with no wrapper catching the non-zero rc to write a marker.

29. user (2026-07-07T14:48:58.110Z)

```
1       # Cloud Run GPU **Job** image for the rally worker (COMPUTE_DECOUPLED_SERVING M6/M7,
ADR-012/D16).
2       #
3       # Runs `python -m backend.worker infer ?` INLINE on an L4 (detect + window + stitch co-
located so
4       # CPU/wall + egress are metered into one job ? D4). Built + pushed + verified by the
owner on the
5       # M7 real run (the $100/?10k budget, D17); CI does NOT build this (a CUDA image is too
heavy) ? the
6       # dispatch shim is proven with a fake client (tests/test_compute_backend.py).
7       #
8       # Two python stacks, deliberately separate (a framework conflict otherwise ? same split
as the
9       # native GPU box, deploy/digitalocean/gpu_setup.sh):
10      #   * the MAIN app env (this repo + ffmpeg) drives the pipeline + stitching;
11      #   * a conda `wasb` env (py3.10 + torch 2.6.0+cu124 + PyNvVideoCodec ? the #506
modernized stack,
12      #     validated on a real L4: cpu-decode F1 0.576 ? the torch-1.11 baseline 0.582) runs
the WASB
13      #     shuttle detector as a subprocess (the native runner shells out to $WASB_PYTHON),
so the app's
14      #     torch never co-installs with it.
15      #
16      # ? WASB-C3: the pretrained badminton weights are academic-data-trained ? provenance NOT
cleared for
17      #   commercial sharing. They are NOT baked into this image; stage them at runtime (a
bucket fetch or a
18      #   mount) and point $WASB_WEIGHTS at them. Resolve WASB-C3 before any public, non-owner
serving.
19
20      # Driver userspace libs for NVDEC/NVENC (#506): GPU hosts inject the COMPUTE driver
stack but not
21      # the video-codec libs ? no libnvcuvid.so.1, no libnvidia-encode.so.1 ? and
PyNvVideoCodec needs
22      # BOTH (it initializes NVENC even for decode-only). Extract the pair from the matching
datacenter
23      # driver .run in a throwaway stage. Keep NVIDIA_DRIVER_VERSION aligned with the GPU
fleet's driver
24      # (580.159.03 = the validated L4 combo); these userspace libs talk to the host-injected
libcuda,
25      # so a large version skew can break decode ? the golden gate on the real image is the
check.
26      ARG NVIDIA_DRIVER_VERSION=580.159.03
27      FROM nvidia/cuda:12.4.1-runtime-ubuntu22.04 AS nvlibs
28      ARG NVIDIA_DRIVER_VERSION
29      RUN apt-get update && apt-get install -y --no-install-recommends curl ca-certificates \
30          && rm -rf /var/lib/apt/lists/* \
31          && curl -fsSL -o /tmp/drv.run \
32              "https://us.download.nvidia.com/tesla/${NVIDIA_DRIVER_VERSION}/NVIDIA-
Linux-x86_64-${NVIDIA_DRIVER_VERSION}.run" \
33          && sh /tmp/drv.run --extract-only --target /tmp/drv \
34          && mkdir -p /nvlibs \
35          && cp -a /tmp/drv/libnvcuvid.so.* /tmp/drv/libnvidia-encode.so.* /nvlibs/ \
36          && rm -rf /tmp/drv /tmp/drv.run
37
38      FROM nvidia/cuda:12.4.1-runtime-ubuntu22.04
39
40      ENV DEBIAN_FRONTEND=noninteractive \
```

```
41          PYTHONUNBUFFERED=1 \
42          PIP_NO_CACHE_DIR=1 \
43          CONDA_DIR=/opt/conda \
44          WASB_REPO_DIR=/opt/models/WASB-SBDT
45
46      # ffmpeg (stitch/decode) + DejaVu fonts (the watermark filter) + git/curl for the WASB
env build.
47      RUN apt-get update && apt-get install -y --no-install-recommends \
48              ffmpeg fonts-dejavu-core git curl ca-certificates build-essential \
49              python3 python3-pip python3-venv \
50          && rm -rf /var/lib/apt/lists/*
51
52      # NVDEC/NVENC userspace libs (#506, from the nvlibs stage above) + the .so.1 sonames
53      # PyNvVideoCodec dlopens. Harmless when decode_backend=cpu; required for nvdec_resident.
54      ARG NVIDIA_DRIVER_VERSION
55      COPY --from=nvlibs /nvlibs/ /usr/lib/x86_64-linux-gnu/
56      RUN ln -sf "libnvcuvid.so.${NVIDIA_DRIVER_VERSION}" /usr/lib/x86_64-linux-
gnu/libnvcuvid.so.1 \
57          && ln -sf "libnvidia-encode.so.${NVIDIA_DRIVER_VERSION}" /usr/lib/x86_64-linux-
gnu/libnvidia-encode.so.1 \
58          && ldconfig
59
60      # --- WASB detector env (micromamba + conda-forge ? BSD-licensed, NO Anaconda ToS; P3a,
ADR-005) ---
61      # Enforces ADR-005 (every dependency ? code, data, weights, AND the build toolchain ?
stays
62      # MIT/Apache/BSD): micromamba (BSD-3) + conda-forge (community) replaces Miniconda's
Anaconda-ToS-gated
63      # `defaults` channels (the ?200-employee Terms-of-Service trap flagged in
PACKAGING_PLAN.md §3).
64      # MAMBA_ROOT_PREFIX=$CONDA_DIR (=/opt/conda) keeps the env at $CONDA_DIR/envs/wasb so
$WASB_PYTHON below
65      # is UNCHANGED.
66      #
67      # #506 modernized stack (validated end-to-end on a real L4, 2026-07-06): py3.10 + torch
2.6.0+cu124
68      # (torch ?2 needs py?3.9; WASB HRNet ports clean ? cpu F1 0.576 ? the torch-1.11
baseline 0.582) +
69      # PyNvVideoCodec (MIT ? the practical NVDEC path; TorchCodec needs a matching ffmpeg
build and could
70      # not load) for decode_backend=nvdec_resident. wasb_infer.py carries the two version
shims this bump
71      # needs (torch.load weights_only=False; the NumPy-2.0 np.Inf alias for WASB's tracker).
numpy is
72      # UNPINNED (the old "numpy<1.24" was the torch-1.11 ABI pin). WASB-SBDT ships no
requirements.txt ?
73      # the explicit list below is the (L4-validated) dependency closure its src/ imports
need.
74      ENV MAMBA_ROOT_PREFIX=/opt/conda
75      RUN curl -sSL https://micro.mamba.pm/api/micromamba/linux-64/latest \
76              | tar -xj -C /usr/local bin/micromamba \
77          && micromamba create -y -q -n wasb -c conda-forge python=3.10 \
78          && git clone --depth 1 https://github.com/nttcom/WASB-SBDT.git "$WASB_REPO_DIR" \
79          && "$CONDA_DIR/envs/wasb/bin/python" -m pip install -q \
80              torch==2.6.0 torchvision==0.21.0 --index-url
https://download.pytorch.org/whl/cu124 \
81          && "$CONDA_DIR/envs/wasb/bin/python" -m pip install -q PyNvVideoCodec==2.1.0 \
82          && "$CONDA_DIR/envs/wasb/bin/python" -m pip install -q \
83              hydra-core omegaconf pandas opencv-python-headless pillow scipy tqdm pyyaml
matplotlib numpy
84
85      # --- Main application env ---
86      WORKDIR /app
87      COPY pyproject.toml README.md ./
88      COPY requirements-trackers.txt ./
```

```
89      COPY backend ./backend
90      COPY training ./training
91      # Install with the [storage-gcs,cloud-run,auth] extras. storage-gcs: the cloud worker
PULLS the gs://
92      # input and PUSHES the gs:// outputs/done-marker through backend.storage (google-cloud-
storage is
93      # lazy-imported there); without it the job dies at the first gs:// fetch with
`ModuleNotFoundError:
94      # google.cloud`. cloud-run: this SAME image is reused as the deployed CONTROL PLANE,
which dispatches
95      # the L4 worker Job via google.cloud.run_v2 (backend/compute/cloud_run.py, lazy-
imported); without it
96      # the control plane fails every dispatch with "google-cloud-run is required for
CloudRunCompute's real
97      # client". Both are real Cloud Run failures the mock-client tests can't see (the SDKs
are faked in CI).
98      # auth: PyJWT[crypto] for M9 edge-auth (security.edge_auth_enabled ? the Cf-Access JWT
verify the
99      # reused-as-control-plane deployment runs behind api.khelsutra.guru); jwt is lazy-
imported and
100     # edge-auth is default-OFF, so the worker path is unaffected ? but without it a control-
plane
101     # deploy with edge-auth ON needed a hand-built derived image (the A1 deploy's PyJWT
overlay).
102     # D13/P12: `trackers` (ByteTrack's new home) is a git-only dep (broken PyPI wheel ?
103     # roboflow/trackers#474) so it's NOT pulled by `pip install .`; install it out-of-band
before
104     # the main env, or the person-cue path crashes with ModuleNotFoundError. The immutable
git SHA is
105     # single-sourced in requirements-trackers.txt.
106     RUN python3 -m pip install --upgrade pip \
107         && python3 -m pip install -r requirements-trackers.txt \
108         && python3 -m pip install ".[storage-gcs,cloud-run,auth]"
109
110     # The native runner finds the WASB env + repo by env (no `conda activate` needed);
weights come in at
111     # runtime (WASB-C3). detector_impl=native = the Linux-native WASB runner; cuda stays the
default.
112     # PYTHONUNBUFFERED: without it the worker's stdout/stderr sit in the block buffer until
process
113     # exit, so Cloud Logging shows a single end-of-run burst instead of live progress
(observed on the
114     # first real M7 runs, 2026-07-03) ? a running job looks stalled and mid-run failures log
nothing.
115     ENV PYTHONUNBUFFERED=1 \
116         WASB_PYTHON=/opt/conda/envs/wasb/bin/python \
117         WASB_REPO_DIR=/opt/models/WASB-SBDT \
118         WASB_WEIGHTS=/opt/models/WASB-SBDT/pretrained_weights/wasb_badminton_best.pth.tar \
119         RALLY_DETECTOR_IMPL=native
120
121     # The dispatcher (CloudRunCompute) supplies the per-execution args:
122     #   infer --video-uri gs://? --out-uri gs://? --done-uri gs://? --segmenter wasb_hybrid
--set ?
123     ENTRYPOINT ["python3", "-m", "backend.worker"]
124
```

30. assistant (2026-07-07T14:49:51.754Z)

I have everything I need. Let me confirm there's no deploy-note/README change in the diff (the mitigation gap) and that the worker uses strict `parse_args`.

31. user (2026-07-07T14:49:54.642Z)

```
 backend/compute/base.py          |  7 +++++++
 backend/compute/cloud_run.py     |  4 ++++
```

```
 backend/orchestration.py         |  7 +++++++
 backend/worker/__main__.py       | 32 ++++++++++++++++++++++++++----
 tests/test_api_cloud_dispatch.py | 16 ++++++++++++++
 tests/test_worker.py             | 45 ++++++++++++++++++++++++++++++++++++++++
 6 files changed, 107 insertions(+), 4 deletions(-)
--- parse_known_args usage? ---
822:    args = build_parser().parse_args(argv)
```

32. assistant (2026-07-07T14:50:11.253Z)

I have a fully verified rollout-skew finding. Summary of what I confirmed by reading the actual code:

- **Deploy model:** the control-plane (Cloud Run *Service*, deployed via `gen_service_yaml.py` + `gcloud run services replace`) and the worker (Cloud Run *Job*, deployed via `gcloud run jobs create/update`) are the *same image* but **independent resources** with independently-pinned image digests. The README §2a already documents that they can skew and warns for #507's `--decode-backend`: "Roll the worker image and the control plane together ? a new control plane forwarding nvdec to an old worker image argparse-fails (rc 2)."
- **The asymmetry vs #509:** #509 forwarded `decode_backend` via the *pre-existing* `--set k=v` arg (orchestration.py:564) ? an old worker never argparse-aborts on it. `--skip-validation` is a *genuinely new argparse flag* (worker `__main__.py:743`, strict `parse_args` at :822).
- **Forward-skew failure path (new control-plane, old worker Job):** `_maybe_dispatch_remote` sets `skip_validation=True` on *every* dispatch (orchestration.py:582) ? cloud_run.py:202 appends `--skip-validation` ? old worker's `main()` `parse_args` aborts `SystemExit(2)` **before** `cmd_infer`/`_fail`/`_write_done_marker` run ? **no done-marker is written**. The dispatcher only bucket-polls the marker (never the Cloud Run API status), so it waits the full `max_wait_sec=1800s` (30 min) ? `PolicyTimeout` ? rc 4 "cloud dispatch error" on every dispatch, including clips the control-plane just passed.
- **Mitigation gap:** the diff ships **no** README/deploy note (unlike #507) and no code guard. A worker-side `parse_known_args` guard added now would not help the already-built old image.
- **Frozen dataclass concern: benign** ? `skip_validation` is a trailing field with a `False` default, `bool` is hashable, all call sites use kwargs (verified all `ComputeJobSpec(` constructions), and the spec is in-memory only (not pickled/persisted cross-version).

33. user (2026-07-07T14:50:27.804Z)

Structured output provided successfully